COMPUTER VISION

Subject Code: 21CSE454T

ASSIGNMENT - UNIT 5

Modern Object Recognition

Task:

Real-world inference application. Load a pre-trained deep learning framework (such as YOLO or MobileNet-SSD) to run real-time multi-class object detection and draw boundary tracking boxes with confidence scores over a custom input video track or webcam stream.

SUBMITTED BY

Name	**:**	Srevarshan
Register Number	**:**	RA2311026050039
Year	**:**	4th Year
Department	**:**	CSE - AIML
Section	**:**	A
Subject	**:**	Computer Vision
Subject Code	**:**	21CSE454T

1. AIM

To load a pre-trained YOLO model and use it to detect and track different types of objects in a video in real time, drawing a bounding box with the class name, track ID and confidence score for each object.

2. ALGORITHM / PROCEDURE

Step 1: Load the pre-trained YOLO11n model. It is trained on the COCO dataset and can detect 80 classes.

Step 2: Open the input video file (or the webcam, using index 0).

Step 3: Read the video one frame at a time.

Step 4: Pass the frame through the YOLO network. In a single pass it predicts the bounding boxes, class labels and confidence scores.

Step 5: Remove the boxes with confidence below 0.35. Duplicate boxes are removed by Non-Maximum Suppression.

Step 6: Use the ByteTrack tracker to match boxes between frames, so each object keeps the same track ID.

Step 7: Draw each box with its class name, track ID and confidence score, and show the FPS.

Step 8: Display or save the frame, and repeat until the video ends (or 'q' is pressed).

Step 9: Print the number of detections per class and the processing speed.

3. PYTHON CODE

File name: unit5_yolo_detection.py

```
"""
Unit 5 - Modern Object Recognition
Real-time multi-class object detection + tracking with a pre-trained YOLO
model (COCO dataset, 80 classes). Draws tracking boxes with class name,
track ID and confidence score on a video file or a live webcam stream.

Usage:
    python unit5_yolo_detection.py --source input_video.webm         # video file
    python unit5_yolo_detection.py --source 0 --show                 # webcam (press q to
        quit)
"""
import os
import time
import argparse
from collections import Counter, defaultdict

import cv2
import numpy as np
import torch
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from ultralytics import YOLO

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "outputs")
os.makedirs(os.path.join(OUT, "examples"), exist_ok=True)

ap = argparse.ArgumentParser()
ap.add_argument("--source", default=os.path.join(HERE, "input_video.webm"), help="video path
    or webcam index")
ap.add_argument("--model", default=os.path.join(HERE, "yolo11n.pt"), help="pre-trained YOLO
    weights")
ap.add_argument("--conf", type=float, default=0.35, help="confidence threshold")
ap.add_argument("--show", action="store_true", help="show live window (press q to quit)")
args = ap.parse_args()

# ---------------- 1. Load the pre-trained model ----------------
DEVICE = 0 if torch.cuda.is_available() else "cpu"
model = YOLO(args.model)                       # downloads the weights the first time
names = model.names
dev_name = torch.cuda.get_device_name(0) if DEVICE == 0 else "CPU"
print(f"Model  : {os.path.basename(args.model)} (COCO, {len(names)} classes) on {dev_name}")

# ---------------- 2. Open the video file / webcam ----------------
source = int(args.source) if str(args.source).isdigit() else args.source
cap = cv2.VideoCapture(source)
assert cap.isOpened(), f"Cannot open source {args.source}"
W, H = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps_in = cap.get(cv2.CAP_PROP_FPS) or 30
total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(f"Source : {os.path.basename(str(args.source))} | {W}x{H} @ {fps_in:.0f} fps | "
      f"frames: {total if total > 0 else 'live'}")
writer = cv2.VideoWriter(os.path.join(OUT, "output_detections.mp4"),
                         cv2.VideoWriter_fourcc(*"mp4v"), fps_in, (W, H))


def color_for(cls_id):
    rng = np.random.default_rng(cls_id * 7 + 3)
    return tuple(int(c) for c in rng.integers(60, 255, 3))


def draw(frame, boxes, fps):
    for (x1, y1, x2, y2), cls, conf, tid in boxes:
        c = color_for(cls)
```

```
        cv2.rectangle(frame, (x1, y1), (x2, y2), c, 2)
        label = f"{names[cls]} #{tid} {conf:.2f}" if tid >= 0 else f"{names[cls]} {conf:.2f}"
        (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
        ty = y1 - 5 if y1 - th - 8 >= 32 else y1 + th + 5          # keep label inside the
            frame
        cv2.rectangle(frame, (x1, ty - th - 3), (x1 + tw + 6, ty + 3), c, -1)
        cv2.putText(frame, label, (x1 + 3, ty), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1,
            cv2.LINE_AA)
    cv2.rectangle(frame, (0, 0), (250, 28), (0, 0, 0), -1)
    cv2.putText(frame, f"FPS: {fps:5.1f} | Objects: {len(boxes)}", (8, 20),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2, cv2.LINE_AA)
    return frame


# ---------------- 3. Detection + tracking loop ----------------
dets = Counter()                                    # detections per class
confs = defaultdict(list)                           # confidence scores per class
ids = defaultdict(set)                              # unique track IDs per class
timeline = []                                       # objects per frame per class
infer_t, examples, best = [], {}, {}
seg_len = max(total // 4, 1) if total > 0 else 300     # one screenshot per quarter of the
    video
frame_i, t_start = 0, time.perf_counter()
while True:
    ok, frame = cap.read()
    if not ok:
        break
    if frame_i == 0:
        cv2.imwrite(os.path.join(OUT, "input_frame.jpg"), frame)   # raw input frame for the
            report
    t0 = time.perf_counter()
    res = model.track(frame, conf=args.conf, persist=True, tracker="bytetrack.yaml",
                      device=DEVICE, verbose=False)[0]
    infer_t.append(time.perf_counter() - t0)

    boxes = []
    if len(res.boxes):
        tids = res.boxes.id.int().tolist() if res.boxes.id is not None else [-1] *
            len(res.boxes)
        for xyxy, cls, conf, tid in zip(res.boxes.xyxy.int().tolist(),
            res.boxes.cls.int().tolist(),
                                        res.boxes.conf.tolist(), tids):
            boxes.append((xyxy, cls, conf, tid))
            dets[names[cls]] += 1
            confs[names[cls]].append(conf)
            if tid >= 0:
                ids[names[cls]].add(tid)
    timeline.append(Counter(names[b[1]] for b in boxes))

    out = draw(frame, boxes, 1 / np.mean(infer_t[-20:]))
    writer.write(out)
    for b in boxes:                                 # first frame where each class appears
        if names[b[1]] not in examples:
            examples[names[b[1]]] = frame_i
            cv2.imwrite(os.path.join(OUT, "examples", f"{names[b[1]].replace(' ', '_')}.jpg"),
                 out)
    score = len(boxes) + 2 * len({b[1] for b in boxes})        # many objects + many classes
    seg = frame_i // seg_len
    if seg not in best or score > best[seg][0]:
        best[seg] = (score, frame_i, out.copy())

    if args.show:
        cv2.imshow("YOLO real-time detection", out)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    frame_i += 1

elapsed = time.perf_counter() - t_start
cap.release(); writer.release(); cv2.destroyAllWindows()
```

```
# Save the busiest frame of each quarter of the video as screenshots
chosen = [(i, img) for _, i, img in sorted(best.values(), key=lambda b: b[1])][:4]
for i, img in chosen:
    cv2.imwrite(os.path.join(OUT, f"frame_{i:04d}.jpg"), img)

# ---------------- 4. Summary ----------------
t = np.array(infer_t[5:])                          # ignore warm-up frames
print(f"\nFrames processed        : {frame_i}")
print(f"Model inference+tracking: {t.mean() * 1000:.1f} ms/frame ({1 / t.mean():.0f} FPS)")
print(f"End-to-end speed        : {frame_i / elapsed:.0f} FPS (read + detect + draw + save)")
print(f"Confidence threshold    : {args.conf}")
print(f"\n{'Class':<12}{'Detections':>11}{'Track IDs':>11}{'Mean conf':>11}{'Max
    conf':>10}{'>=0.5':>7}")
print("-" * 62)
for cls, n in dets.most_common():
    c = np.array(confs[cls])
    print(f"{cls:<12}{n:>11}{len(ids[cls]):>11}{c.mean():>11.3f}{c.max():>10.3f}{(c >=
        0.5).sum():>7}")
print(f"\nFirst frame of each class: {examples}")
print(f"Screenshots saved (frames): {sorted(i for i, _ in chosen)}")

# Timeline plot: number of objects of each class in every frame
fig, ax = plt.subplots(figsize=(10, 3.4))
k = max(int(fps_in // 2), 1)                       # 0.5 s moving average
for cls, _ in dets.most_common(4):
    counts = np.array([tl[cls] for tl in timeline], float)
    ax.plot(np.convolve(counts, np.ones(k) / k, mode="same"), lw=1.6, label=cls)
ax.set_xlabel("Frame number"); ax.set_ylabel("Objects detected"); ax.legend(ncol=4)
ax.set_title("Objects detected per frame (0.5 s moving average)")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "timeline.png"), dpi=120); plt.close()
print(f"Annotated video saved   : outputs/output_detections.mp4")
```

4. INPUT IMAGE



Figure 1: Input video (first frame) - street crossing, 854 x 480, 30 fps, 31 seconds. Source: Wikimedia Commons, by RG72, CC BY-SA 4.0

5. OUTPUT SCREENSHOTS

```
Terminal

$ python unit5_yolo_detection.py
Model  : yolo11n.pt (COCO, 80 classes) on NVIDIA GeForce RTX 4050 Laptop GPU
Source : input_video.webm | 854x480 @ 30 fps | frames: 940

Frames processed        : 917
Model inference+tracking: 6.8 ms/frame (146 FPS)
End-to-end speed        : 87 FPS (read + detect + draw + save)
Confidence threshold    : 0.35

Class        Detections  Track IDs  Mean conf  Max conf  >=0.5
------------------------------------------------------------
person             4308        109      0.536     0.845   2179
car                3825        138      0.652     0.943   2665
traffic light      1350         22      0.577     0.802    883
handbag             656          4      0.486     0.661    303
backpack              1          1      0.391     0.391      0
umbrella              1          1      0.364     0.364      0

First frame of each class: {'person': 0, 'traffic light': 0, 'car': 0, 'handbag': 0, 'backpack': 254, 'umbrella': 353}
Screenshots saved (frames): [222, 386, 514, 710]
Annotated video saved   : outputs/output_detections.mp4
```

Figure 2: Program output in the terminal

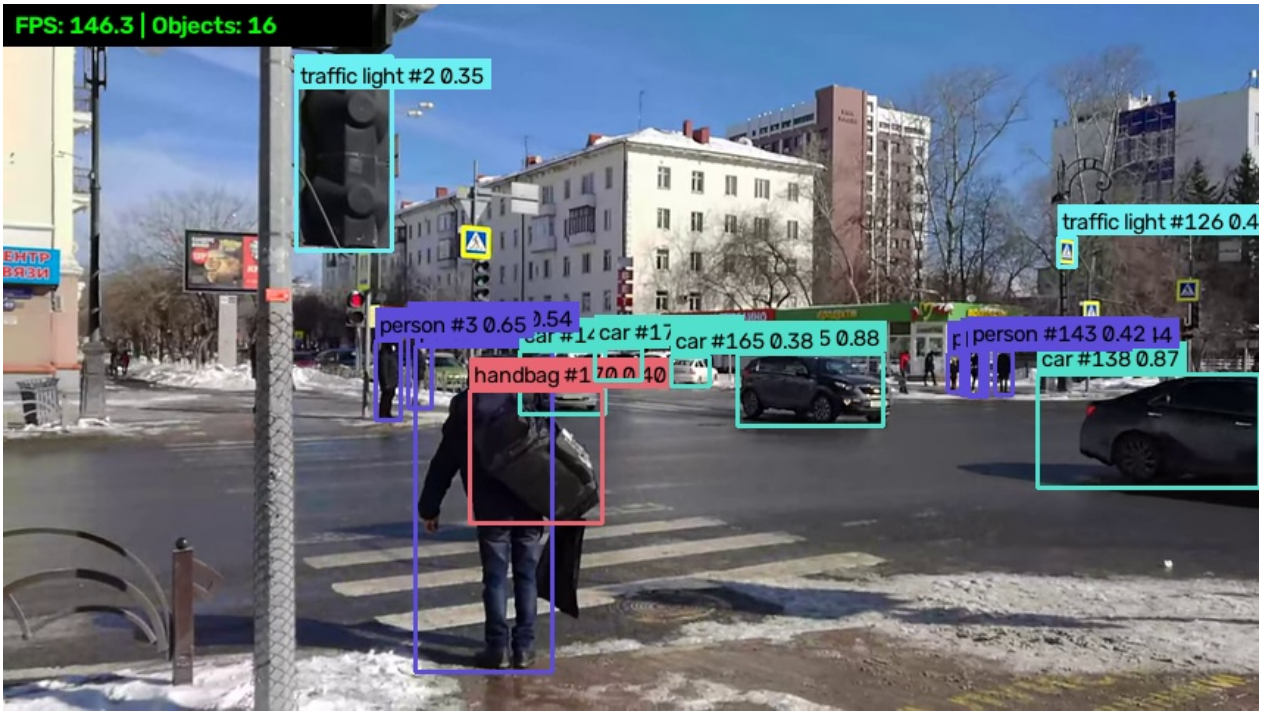


Figure 3: Detection output - frame 222

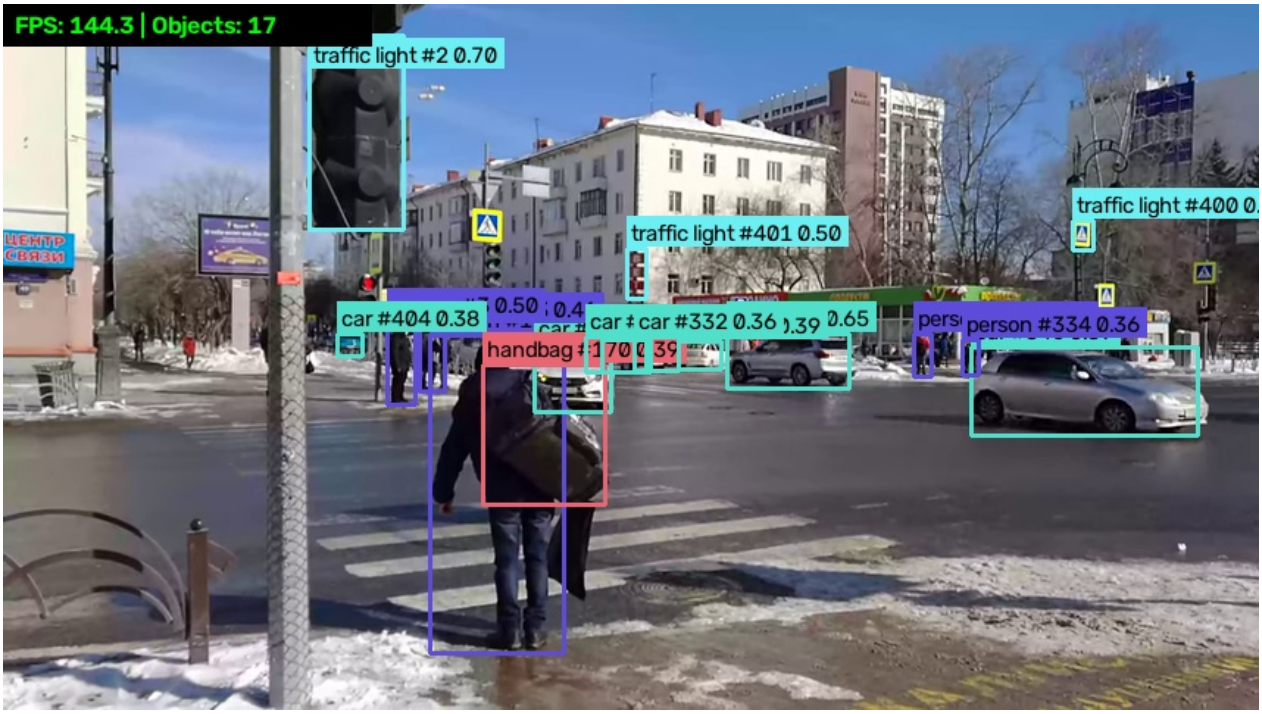


Figure 4: Detection output - frame 514

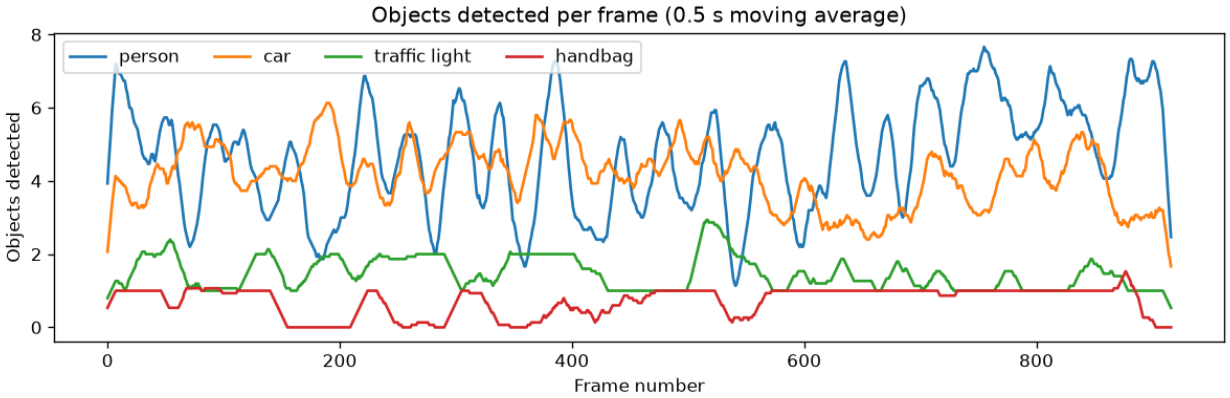


Figure 5: Number of objects of each class detected over the video

6. RESULT ANALYSIS / OBSERVATIONS

Item	Value
Model	YOLO11n (pre-trained on COCO, 80 classes)
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Frames processed	917
Detection + tracking time	6.8 ms per frame (146 FPS)
Overall speed (read, detect, draw, save)	87 FPS

Class	Detections	Mean confidence	Track IDs
person	4308	0.536	109
car	3825	0.652	138
traffic light	1350	0.577	22
handbag	656	0.486	4
backpack	1	0.391	1
umbrella	1	0.364	1

- The program ran at 146 FPS (87 FPS including reading, drawing and saving). This is faster than the 30 fps video, so the detection works in real time.
- Four classes were detected in almost every frame: person, car, traffic light and handbag. Cars had the highest average confidence (0.65).
- Nearby objects were detected with high confidence (up to 0.94). Small and distant people and cars got lower scores, between 0.35 and 0.5.
- Backpack and umbrella were detected only once, with low confidence. These are most likely wrong detections and would be removed by a threshold of 0.5.
- There are more track IDs than real objects (for example 109 for person). A new ID is given when a person is hidden behind another object or is too far away to be detected for a few frames.

7. CONCLUSION

A pre-trained YOLO11n model detected and tracked people, cars, traffic lights and handbags in a street video in real time, at 146 FPS on the GPU. Each object was shown with a bounding box, track ID and confidence score. Pre-trained deep learning models can be used for real-world detection without any training. However, small or distant objects get lower confidence, and the confidence threshold must be chosen carefully to remove false detections.